

Chapter 5

Evolving Spiking Neural Networks

Evolving SNN (eSNN) are a class of SNN and also a class of ECOS (Chapter 2) where spiking neurons are created (evolved) and merged in an incremental way to capture clusters and patterns from incoming data. This gives a new quality of the SNN systems to become adaptive, fast trained and to capture meaningful patterns from the data, departing the “curse of the black box neural networks” and the “curse of catastrophic forgetting” as manifested by some traditional ANN models (Chapter 2). The inspiration comes from the brain as the brain always evolves its structure and functionality through continuous learning. It is always *evolving* and forming new knowledge.

The chapter is organised in the following sections:

- 5.1. Principles and methods of eSNN
- 5.2. Convolutional eSNN
- 5.3. Dynamic eSNN (deSNN)
- 5.4. Fuzzy rule extraction from eSNN
- 5.5. Evolving SNN for reservoir computing
- 5.6. Chapter summary and further readings for deeper knowledge

5.1. Principles and methods of evolving SNN (eSNN)

The eSNN paradigm applies the ECOS principles (Chapter 2) to process spike-time information, namely:

- (1) Fast learning from large amount of data, e.g. using “one-pass” training, starting with little prior knowledge;
- (2) Adaptation in real-time and in an on-line mode where new data is accommodated as it comes based on local learning;
- (3) “Open”, evolving structure, where new input variables (relevant to the task), new outputs (e.g. classes), new connections and neurons are added/evolved “on the fly”;

- (4) Both data learning and knowledge representation is facilitated in a comprehensive and flexible way, e.g., supervised learning, unsupervised learning, evolving clustering, “sleep” learning, forgetting/pruning, fuzzy rule insertion and extraction;
- (5) Active interaction with other systems and with the environment in a multi-modal fashion;
- (6) Representing both space and time in their different scales, e.g., clusters of data, short- and long-term memory, forgetting, etc.;
- (7) System’s self-evaluation in terms of behavior, global error and success, and related knowledge representation.

There are several models of eSNN models.

The simplest eSNN mode uses (see Chapter 4):

- Population spike coding algorithm (POC);
- Leaky integrate-and fire (LIF) model of a neuron;
- Rank-order (RO) learning rule.

This is schematically shown in Fig. 1a,b.

The RO learning motivation is based on the assumption that most important information of an input pattern is contained in earlier arriving spikes [5]. It establishes a priority of inputs based on the order of the spike arrival on the input synapses for a particular pattern. This is a phenomenon observed in biological systems as well as an important information processing concept for some spatio-temporal problems, such as computer vision and control. RO learning makes use of the information contained in the order of the input spikes (events). This method has two main advantages when used in SNN: (1) fast learning (as the order of the first incoming spikes is often sufficient information for recognising a pattern and for a fast decision making and only one pass propagation of the input pattern may be sufficient for the model to learn it); (2) asynchronous, data-driven processing. As a consequence, RO learning is most appropriate for AER input data streams as the address-events are conveyed into the SNN ‘one by one’, in the order of their happening [65, 66]. The RO coding for the eSNN structure from Fig.1 is illustrated in Chapter 4.

An eSNN evolves its structure and functionality in an on-line manner, from incoming information. For every new input data vector, a new output neuron is dynamically allocated and connected to the input neurons (feature neurons). The neuron’s connections are established using the RO rule for the output neuron to recognise this vector (frame, static pattern) or a similar one as a positive example.

The weight vectors of the output neurons represent centres of clusters in the problem space and can be represented as fuzzy rules [67].

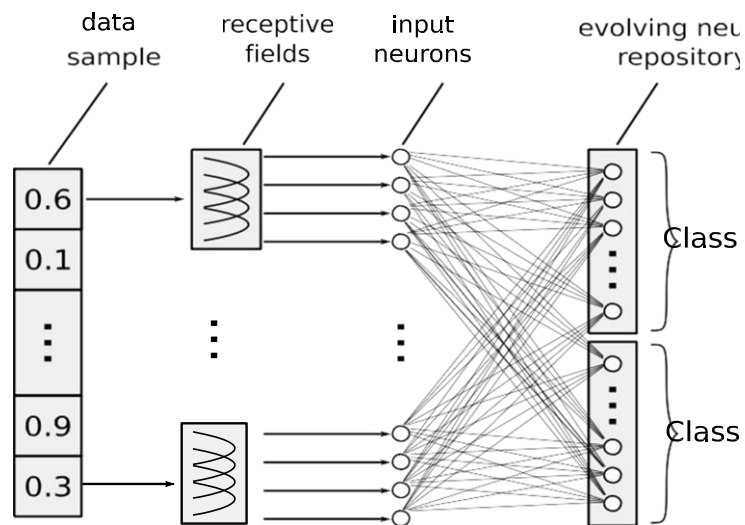
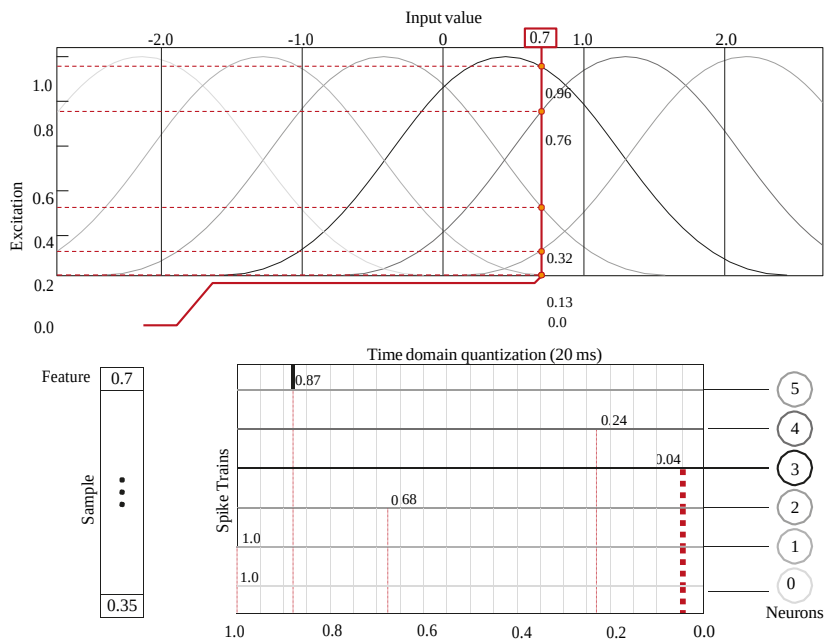


Figure 1a. A principle diagram of the structure of an evolving SNN (eSNN) where the input variables are encoded into spike sequences through population coding (POC) and the output neuronal layer evolves with the presentation of every new data input vector, allowing also for merging of output neurons.



Firing time (1 - Excitation)

Figure 1b: Example of how a random continuous value is encoded into spike trains using RO coding for a number of input neurons. The value of 0.7 activates 6 overlapping receptive fields, which excite 6 corresponding input neurons at different firing times.

In some implementations neurons with similar weight vectors are merged based on Euclidean distance between them. That makes it possible to achieve a very fast learning (only one pass may be sufficient), both in a supervised and in an unsupervised mode [68]. When in an unsupervised mode, the evolved neurons represent a learned pattern (or a prototype of patterns). The neurons can be labelled and grouped according to their belonging to the same class if the model performs a classification task in a supervised mode of learning – an example is shown in fig.25.

During a *learning phase*, for each M-dimensional training input pattern (sample, example, vector) P_i a new output neuron i is created and its connection weights $w_{j,i}$ ($j=1,2,...,M$) to the input (feature) neurons are calculated based on the order of the incoming spikes on the corresponding synapses using the RO learning rule :

$$w_{j,i} = \alpha \cdot \text{mod}^{\text{order}(j,i)} \quad (1)$$

where: α is a learning parameter (in a partial case it is equal to 1); *mod* is a modulation factor, that defines how important the order of the first spike is; $w_{j,i}$ is the synaptic weight between a pre-synaptic neuron j and the postsynaptic neuron i ; *order*(j,i) represents the order (the rank) of the *first* spike at synapse j,i ranked among all spikes arriving from all synapses to the neuron i ; *order*(j,i) has a value 0 for the first spike to neuron i and increases according to the input spike order at other synapses.

While the input training pattern (example) is presented (all input spikes on different synapses, encoding the input vector are presented within a time window of T time units), the spiking threshold Th_i of the neuron i is defined to make this neuron spike when this or a similar pattern (example) is presented again in the recall mode. The threshold is calculated as a fraction (C) of the total PSP_i (denoted as $PSP_{i\max}$) accumulated during the presentation of the input pattern:

$$PSP_i^{\max} = \sum_j \text{mod}^{\text{order}(j,i)} \quad (2)$$

$$\Theta = CPSP_i^{\max} \quad (3)$$

If the weight vector of the evolved and trained new neuron is similar to the one of an already trained neuron (in a supervised learning mode for classification this is a neuron from the same class pool), i.e. their similarity is above a certain threshold *Sim*, the new neuron will be merged with the most

similar one, averaging the connection weights and the threshold of the two neurons [68, 69]. Otherwise, the new neuron will be added to the set of output neurons (or the corresponding class pool of neurons when a supervised learning for classification is performed). The similarity between the newly created neuron and a training neuron is computed as the inverse of the Euclidean distance between weight matrices of the two neurons. The merged neuron has weighted average weights and thresholds of the merging neurons.

While an individual output neuron represents a single input pattern, merged neurons represent clusters of patterns or prototypes in a transformed spatial – RO space. These clusters can be represented as fuzzy rules [67] that can be used to discover new knowledge about the problem under consideration.

The eSNN learning is adaptive, incremental, theoretically – ‘lifelong’, so that the system can learn new patterns through creating new output neurons, connecting them to the input neurons, and possibly merging the most similar ones, following the ECOS principles from Chapter 2.

During the *recall phase*, when a new input vector is presented and encoded as input spikes, the spiking pattern is submitted to all created neurons during the learning phase. An output spike is generated by neuron i at a time l if the $PSP_i(l)$ becomes higher than its threshold Θ . After the first neuron spikes, the PSP of all neurons are set to initial value (e.g. 0) to prepare the system for the next pattern for recall or learning.

The postsynaptic potential $PSP_i(l)$ of a neuron i at time l is calculated as:

$$PSP_i(l) = \sum_{t=0,1,2,\dots,l} \sum_j e_j(t) \cdot \text{mod}^{\text{order}(j,i)} \quad (3)$$

where: $e_j(t)=1$ if there is a *first* spike at time t on synapse j ; *order* (j,i) is the rank order of the first spike at synapse j among all spikes to neuron i for this recall pattern.

The parameter C , used to calculate the threshold of a neuron i , makes it possible for the neuron i to emit an output spike before the presentation of the whole learned pattern (lasting T time units) as the neuron was initially trained to respond. As a partial case $C=1$.

During training of an eSNN, for each input vector the following steps are performed:

- a) Create (evolve) a new output spiking neuron and its connections;
- b) Propagate the input vector into the network calculating the PSP of the output neurons:

$$u_i(t) = \begin{cases} 0 & \text{if fired} \\ \sum_{j|f(j)<t} w_{ji} m_i^{\text{order}(j)} & \text{else} \end{cases} \quad (4)$$

- c) Train the newly created neuron weights change based on the spike time arrival.

d) IF similarity between a new and old neurons is greater than a Threshold THEN merge neurons,
where N is the number of samples previously used to update the respective neuron.

d) Update the corresponding threshold ϑ of the merged neuron

The eSNN training algorithm is given in Box 1.

Box 1. The eSNN training algorithm

```

1: Initialize output neuron repository,  $R = \{\}$ 
2: Set eSNN parameters:  $mod = [0, 1]$ ,  $C = [0, 1]$ ,  $sim = [0, 1]$ 
3: for  $\forall$  input pattern  $i$  that belongs to the same class do
4:   Encode input pattern into firing time of multiple pre-synaptic neurons  $j$ 
5:   Create a new output neuron  $i$  for this class and calculate the connection weights as  $w_{ji} = mod^{order(j)}$ 
6:   Calculate  $PSP_{max(i)} = \sum_j w_{ji} \times mod^{order(j)}$ 
7:   Calculate  $PSP$  threshold value  $\gamma_i = PSP_{max(i)} \times C$ 
8:   if The new neuron weight vector  $\leq sim$  of trained output neuron weight vector in  $R$  then
9:     Update the weight vector and threshold of the most similar neuron in the same output class group
10:     $w = (w_{new} + w * N) / N + 1$ 
11:     $\gamma = \frac{\gamma_{new} + \gamma * N}{(N+1)}$ , where  $N$  is the number of previous merges of the most similar neuron
12:  else
13:    Add the weight vector and threshold of the new neuron to the neuron repository  $R$ 
14:  end if
15: end for
16: end for
17: Repeat above for all input patterns of other output classes

```

The recall procedure can be performed using different recall algorithms implying different methods of comparing input patterns for recall with already learned patterns in the output neurons:

(a) The first one is described above. Spikes of the new input pattern are propagated as they arrive to all trained output neurons and the first one that spikes (its PSP is greater than its threshold) defines the output. The assumption is that the neuron that best matches the input pattern will spike earlier based purely on the PSP (membrane potential). This type of eSNN is denoted as eSNNm.

- (b) The second one implies a creation of a new output neuron for each recall pattern, in the same way as the output neurons were created during the learning phase, and then – comparing the connection weight vector of the new one to the already existing neurons using Euclidean distance. The closest output neuron in terms of synaptic connection weights is the ‘winner’. This method uses the principle of transductive reasoning and nearest neighbour classification in the connection weight space. It compares spatially distributed synaptic weight vectors of a new neuron that captures a new input pattern and existing ones. We will denote this model as eSNNs.

The main advantage of the eSNN when compared with other supervised or unsupervised SNN models are:

- It is computationally inexpensive;
- It boosts the importance of the order in which input spikes arrive, thus making the eSNN suitable for a range of applications.
- It is one-pass, on-line learning method, where new data can be learned incrementally in a “life-long:” learning method, involving the merging/aggregating output neurons;
- It is knowledge-based, where the output neurons (after aggregation) represent prototypes in the data or cluster centres;
- Allow for fuzzy rule extraction as presented in a later section.

For a comprehensive study of eSNN see [69] and for a comprehensive review - [7].

The problem of the eSNN is that once a synaptic weight is calculated based on the first spike using the RO rule, it is fixed and does not change to reflect on other incoming spikes at the same synapse, i.e. there is no mechanism to deal with multiple spikes arriving at different times on the same synapse. The synapses are static. While the synapses capture some (long term) memory during the learning phase, they have limited abilities (only through the PSP growth) to capture short term memory during a whole spatio-temporal pattern presentation. Learning and recall of complex spatio-temporal patterns in an on-line mode would need not only fast initial set of connection weights, based on the first spikes, but also dynamic changes of these synapses during the pattern presentation.

An example of an eSNN is shown in Fig.2 [70]:

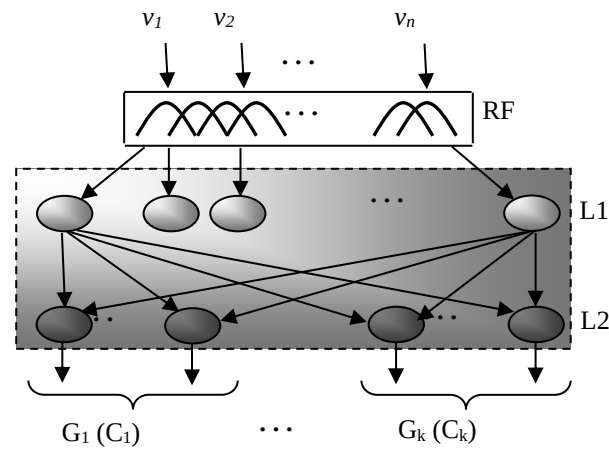


Figure 2. An example of an eSNN structure with n input variables and k output classes

5.2. Convolutional eSNN (CeSNN)

Here, eSNN used to build convolutional eSNN (CeSNN). The principles of convolutional NN was presented and illustrated in Chapter 2. Here CNN is created using the eSNN model and illustrated on image recognition problem as shown in Fig.3.

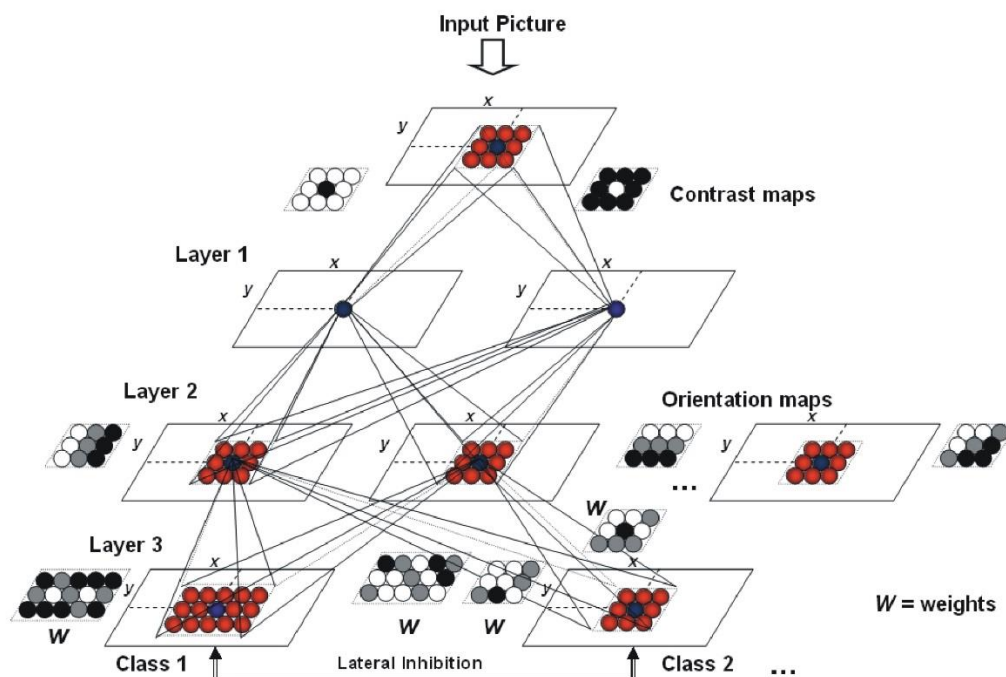


Figure 3. A convolutional eSNN for image pattern recognition

The neural network is composed of 3 layers of integrate-and-fire neurons. The neurons have a latency of firing that depends upon the order of spikes received. Each neuron acts as a coincidence detection unit, where the postsynaptic potential for neuron N_i at a time t is calculated as:

$$PSP(i, t) = \sum \text{mod}^{\text{order}(j)} w_{j,i} \quad (5)$$

where $\text{mod} \in (0,1)$ is the modulation factor, j is the index for the incoming connection and $w_{j,i}$ is the corresponding synaptic weight.

Each layer is composed of neurons that are grouped in two-dimensional grids forming neuronal maps. Connections between layers are purely feed-forward and each neuron can spike at most once on a spike arrival in the input synapses. The first layer cells represent the ON and OFF cells of retina, basically enhancing the high contrast parts of a given image (high pass filter). The output values of the first layer are encoded to pulses in the time domain. High output values of the first layer are encoded as pulses with short time delays while long delays are given low output values. This technique is called Rank Order Coding (Thorpe et al, 1998) and basically prioritizes the pixels with high contrast that consequently are processed first and have a higher impact on neurons' PSP.

Second layer is composed of eight orientation maps, each one selective to a different direction (0° , 45° , 90° , 135° , 180° , 225° , 270° , and 315°). It is important to notice that in the first two layers there is no learning, in such a way that the structure can be considered simply passive filters and time domain encoders (layers 1 and 2). The theory of contrast cells and direction selective cells was first reported by Hubel and Wiesel (1962). In their experiments they were able to distinguish some types of cells that have different neurobiological responses according to the pattern of light stimulus.

The third layer is where the learning takes place and where the main contribution of this work is presented. Maps in the third layer are to be trained to represent classes of inputs. In (Thorpe et al. 1998) the learning is performed off-line using the rule:

$$\Delta w_{j,i} = \frac{\text{mod}^{\text{order}(a_j)}}{N} \quad (6)$$

where $w_{j,i}$ is the weight between neuron j of the 2nd layer and neuron i of the 3rd layer, $\text{mod} \in (0,1)$ is the modulation factor, $\text{order}(a_j)$ is the order of arrival of spike from neuron j to neuron i , and N is the number of samples used for training a given class.

In this rule, there are two points to be highlighted: a) the number of samples to be trained needs to be known *a priori*; and b) after training, a map of a class will be selective to the average pattern.

In [69] a new approach is proposed for learning with structural adaptation, aiming to give more flexibility to the system in a scenario where the number of classes and/or class instances is not known at the time the training starts. Thus, the output neuronal maps need to be created, updated or even deleted on-line, as the learning occurs. To implement such a system the learning rule needs to be independent of the total number of samples since the number of samples is not known when the learning starts.

The entire training procedure for the CeSNN from Fig. 3 follows steps described next and summarized in the flowchart of Fig. 4.

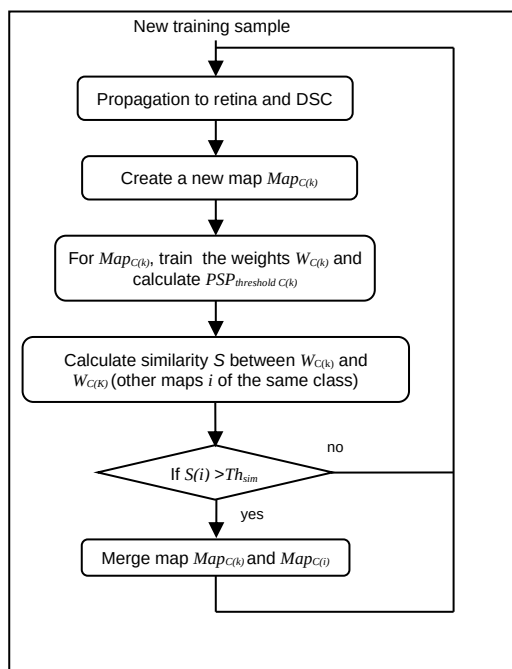


Figure 4. The training procedure for the CeSNN from Fig. 3

1. Propagate a sample k of class K for training into the layer 1 (retina) and layer 2 (direction selective cells – DSC);
2. Create a new map $Map_{C(k)}$ in layer 3 for sample k and train the weights using the equation:

$$\Delta w_{j,i} = \text{mod}^{order(a_j)} \quad (7)$$

where $w_{j,i}$ is the weight between neuron j of the layer 2 and neuron i of the layer 3, $\text{mod} \in (0,1)$ is the modulation factor, $order(a_j)$ is the order of arrival of spike from neuron j to neuron i .

The postsynaptic threshold ($PSP_{threshold}$) of the neurons in the map is calculated as a proportion $c \in [0,1]$ of the maximum postsynaptic potential (PSP) created in a neuron of map $Map_{C(k)}$ with the propagation of the training sample into the updated weights, such that:

$$PSP_{threshold} = c \max(PSP) \quad (8)$$

The constant of proportionality c express how similar a pattern needs to be to trigger an output spike. Thus, c is a parameter to be optimized in order to satisfy the requirements in terms of false acceptance rate (FAR) and false rejection rate (FRR).

3. Calculate the similarity between the newly created map $Map_{C(k)}$ and other maps belonging to the same class $Map_{C(K)}$. The similarity is computed as the inverse of the Euclidean distance between weight matrices.
4. If one of the existing maps for class K has similarity greater than a chosen threshold $Th_{simC(K)} > 0$, merge the maps $Map_{C(k)}$ and $Map_{C(Ksimilar)}$ using arithmetic average as expressed in equation

$$W = \frac{W_{Map_{C(k)}} + N_{samples} W_{Map_{C(Ksimilar)}}}{1 + N_{samples}} \quad (9)$$

where matrix W represents the weights of the merged map and $N_{samples}$ denotes the number of samples that have already being used to train the respective map. In similar fashion the $PSP_{threshold}$ is updated:

$$PSP_{threshold} = \frac{PSP_{Map_{C(k)}} + N_{samples} PSP_{Map_{C(Ksimilar)}}}{1 + N_{samples}} \quad (10)$$

The convolutional eSNN first extract features from the input data through the convolutional layers. When input vectors are presented one after another, spikes are accumulated over time in the membrane potentials of the output neurons until an output neuron spikes identifying the class output. Then the membrane potentials are set to resting potential for the next input vectors to be received and classified. A convolutional eSNN for person authentication based on both speech and face data is presented in Fig. 5 and described in details in Chapter 12. [69].

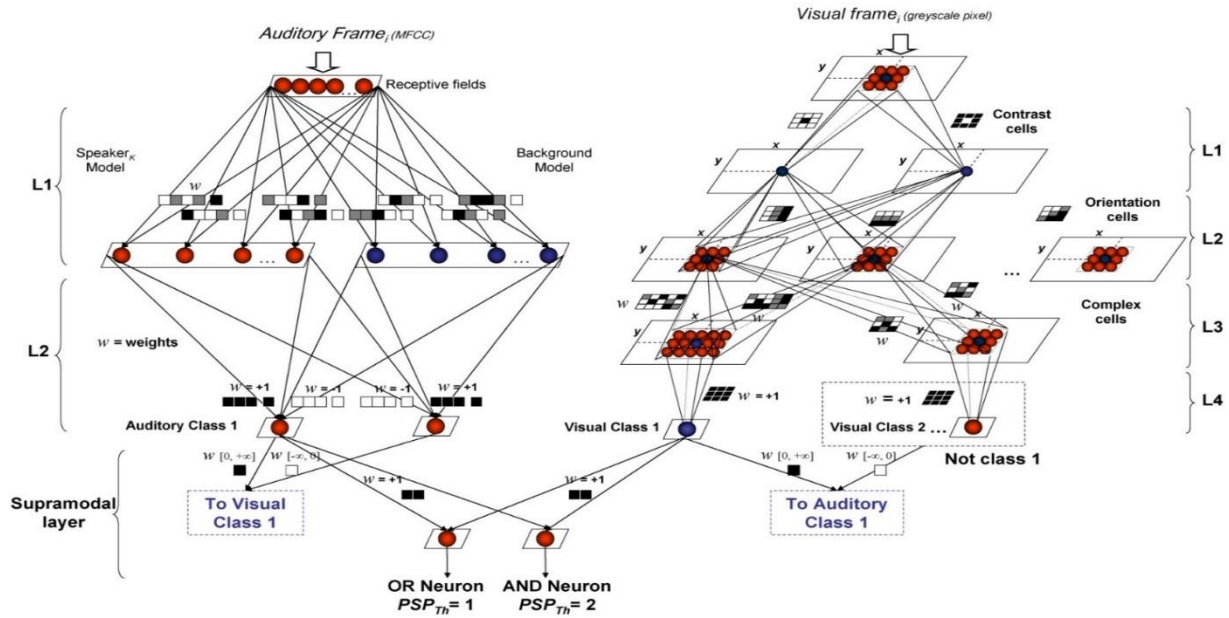


Figure 5. CeSNN for person identification using both speech and face image input data [69]

5.3. Dynamic Evolving SNN (deSNN)

This method is first presented in [41]. The main disadvantage of the RO learning in eSNN is that the model adjusts the connection weight of each synapse once only (based on the rank of the first spike on this synapse), which may be appropriate for static pattern recognition, but would not be efficient for complex SSTD. In the latter case the connection weights need to be further tuned based on the following spikes arriving on the same synapse over time and that is where the spike-time learning (e.g. STDP or SDSP) can be employed in order to implement dynamic synapses.

In the proposed deSNN both the RO and the SDSP learning rules are utilised. While the RO learning will set the initial values of the connection weights $\mathbf{w}(0)$ (utilising for example the existing event order information in an AER data), the SDSP rule will adjust these connection weights based on further incoming spikes (events) as part of the same learned spatio-temporal pattern.

As in the eSNN, during a *learning phase*, for each training input pattern (sample, example, vector) P_i a new output neuron i is created and its connection weights $w_{j,i}$ to the input (feature) neurons are initially calculated as $w_{j,i}(0)$ based on the order of the incoming spikes on the corresponding synapses using the RO learning rule.

Once a synaptic weight $w_{j,i}$ is initialised, based on the first spike at the synapse j , the synapse becomes dynamic and adjusts its weight through the SDSP algorithm. It increases its value with a small positive value (positive drift parameter) at any time t a new spike arrives at this synapse and decreases its value (a negative drift parameter) if there is no spike at this time.

$$\Delta w_{j,i}(t) = e_j(t) \cdot D \quad (11)$$

where: $e_j(t) = 1$ if there is a consecutive spike at synapse j at time t during the presentation of the learned pattern by the output neuron i and (-1) otherwise. In general, the drift parameter D can be different for ‘up’ and ‘down’ drifts.

All dynamic synapses change their values in parallel for every time unit t during a presentation of an input spatio-temporal pattern P_i learned by an output neuron i , some of them going up and some – going down, so that all synapses (not a single one) of the neuron could collectively capture some temporal relationship of spike timing across the learned pattern.

While an input training pattern (example) is presented (all input spikes on different synapses, encoding the input vector are presented within a time window of T time units), the spiking threshold Th_i of the neuron i is defined to make this neuron spike when this or a similar pattern (example) is

presented in the recall mode. The threshold is calculated as a fraction (C) of the total PSP_i (denoted as PSP_{imax}) accumulated during the presentation of the whole input pattern:

$$PSP_{imax} = \sum_{t=1,2,\dots,T} \sum_{j=1,2,\dots,M} f_j(t) \cdot w_{j,i}(t) \quad (12)$$

$$Th_i = C \cdot PSP_{imax} \quad (13)$$

where: T represents the time units in which the input pattern is presented; M is the number of the input synapses to neuron i ; $f_j(t)=1$ if there is spike at time t at synapse j for this learned input pattern, otherwise it is 0; $w_{j,i}(t)$ is the efficacy of the (dynamic) synapse between j and i neurons calculated at time t with the use of formula (50).

The resulted deSNN model after training will contain the following information:

- Number of input neurons M and output neurons N ;
- Initial $\mathbf{w}_i(0)$ and final $\mathbf{w}_i(T)$ vectors of connection weights and spiking threshold Th_i for each of the output neurons i . The pairs $[\mathbf{w}_i(0), \mathbf{w}_i(T)]$, $i=1,2,\dots,N$ would capture collectively dynamics of the learning process for each spatio-temporal pattern and each output neuron (As a partial case only initial or final values of the connection weights can be considered or a weighted sum of them).
- deSNN parameters.

The overall deSNN training algorithm is presented in Box 2.

Box 2. The deSNN Training Algorithm

- 1 Set deSNN parameters* (including: Mod, C, Sim, and the SDSP parameters)
- 2 **For** every input spatio-temporal spiking pattern P_i **Do**
 - 2a. Create a new output neuron i for this pattern and calculate the initial values of connection weights $\mathbf{w}_i(0)$ using the RO learning formula (1).
 - 2b. Adjust the connection weights $\mathbf{w}_i$ for consecutive spikes on the corresponding synapses using the SDSP learning rule (formula (7)).
 - 2c. Calculate PSP_{imax} using formula (8).
 - 2d. Calculate the spiking threshold of the i^{th} neuron using formula (9).

2e. **(Optional)** If the new neuron weight vector $\mathbf{w}_i$ is similar in its initial $\mathbf{w}_i(0)$ and final $\mathbf{w}_i(T)$ values after training to the weight vector of an already trained output neuron using Euclidean distance and a similarity threshold Sim , then merge the two neurons (as a partial case only initial or final values of the connection weights can be considered or a weighted sum of them)

Else

Add the new neuron to the output neurons repository.

End If

End For (Repeat for all input spatio-temporal patterns for learning)

*: The performance of the deSNN depends on the optimal selection of its parameters as illustrated in the examples below.

Fig. 6 illustrates the main idea of the deSNN learning algorithm. A single spatio-temporal pattern of four input spike trains is learned into a single output neuron. RO learning is applied to calculate the initial weights based on the order of the first spike on each synapse (shown in red). Then STDP (in this case – SDSP) rule is applied to dynamically tune these connection weights. The SDSP algorithm increases the assigned connection weight of a synapse which is receiving a following spike and at the same time depresses the synaptic connections of synapses that do not receive a spike at this time. Due to a bi-stability drift in the SDSP rule, once a weight reaches the defined High value (resulting in LTP) or Low value (resulting in LTD), this connection weight is fixed to this value for the rest of the training phase. The rate at which a weight reaches LTD or LTP depends upon the set parameter values.

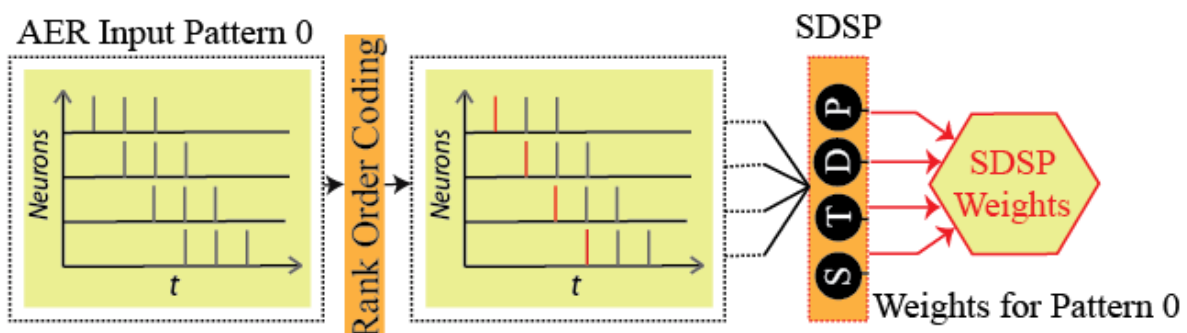


Figure 6. A simple example to illustrate the main principle of the deSNN learning algorithm.

For example, if input spikes arrive at times (0,1,2) ms on the first synapse, and are shifted by 1ms for the other 3 synapses as shown in fig.4, the four initial connection weights w_1, w_2, w_3, w_4 to the output neuron will be calculated as: 1, 0.8, 0.64, 0.512 correspondingly, when the parameter $\text{mod}=0.8$. If the SDSP High value is 0.6 and Low value is 0, the first three weights will be fixed to 0.6 and the fourth one will drift up 2 times. If the drift parameter is set to 0.00025, the final weight value of the fourth synapse will be 0.5125. After training both the initial and the final weights can be memorised.

So far, we have presented the learning phase of a deSNN model. In terms of *recall*, two types of deSNN are proposed that differ in the recall algorithms. They mainly correspond to the two types of eSNN from section 2- eSNNs and eSNNm:

(a) deSNNm: After learning, only the initially created connection weights (with the use of the RO rule) are restored as long term memory in the synapses and the model. During recall on a new spatio-temporal pattern the SDSP rule is applied so that the initial synaptic weights are modified on a spike time basis according to the new pattern using formula (50) as it is during the SDSP learning phase. At every time moment t the PSP of all output neurons are calculated. The new input pattern is associated with the neuron i if the $PSP_i(t)$ is above its threshold Th_i . The following formula is used:

$$PSP_i(t) = \sum_{l=1,2,\dots,t} \sum_{j=1,2,\dots,M} f_j(l) \cdot w_{j,i}(l) \quad (14)$$

where: t represents the current time unit during the presentation of the input pattern for recall; M is the number of the input synapses to neuron i ; $f_j(l)=1$ if there is spike at time l at synapse j for this input pattern, otherwise it is 0; $w_{j,i}(l)$ is the efficacy of the dynamic synapse between j and i neurons at time l .

(b) deSNNs: This model corresponds to the eSNNs and is based on the comparison between the synaptic weights of a newly created neuron to represent the new spatio-temporal pattern for recall, and the connection weights of the created during training neurons. The new input pattern is associated with the closest output neuron based on the minimum distance between the weight vectors. As the synaptic weights are dynamic, the distance should be calculated in a different way than the distance measured in the eSNN possibly using both the initial $\mathbf{w}(0)$ and the final $\mathbf{w}(T)$ connection weight vectors learned during training and recall. As a partial case, only the final weight vector $\mathbf{w}(T)$ can be used.

To summarise, deSNN combine:

- RO learning for weight initialisation based on the first spikes:
- Learning further input spikes at a synapse through a drift.
 - $w_{j,i}(t) = e_j(t) \cdot \text{Drift}$ (15)
- A new output neuron may be added to a respective output repository for every new -input pattern.
- Two types of output neuron activation:
 - deSNNm (spiking is based on the membrane potential)
 - deSNNs (spiking is based on synaptic similarity between the newly created output neuron and the existing ones)
- Neurons may merge.

5.4. Fuzzy rule extraction from eSNN

One of the challenge with SNN is the difficulty to make sense out of the learned connections as they evolve from sequences of spikes. What do actually connections represent? Do they represent any meaningful patterns that can be interpreted for a better understanding of the data and the processes that generated it?

This section addresses this challenge by introducing a method for fuzzy rule extraction from eSNN.

5.4.1. Fuzzy rule extraction from eSNN

The rationale behind the presented here method for fuzzy rule extraction from eSNN is that input information is represented as spikes in time that account for the intensity of the input variables through population spike coding algorithm POC. The higher the membership degree of an input variable to a neuronal receptive field in layer L1 of neurons (see Fig.2), the earlier a spike is generated. The connection weights learned from these spikes in the L2 layer using ROC increase more the earlier spikes are generated in layer L1. So, in a sense the connection weights from layer L1 to L2 reflect on the values of the input variables and their membership degrees to the different receptive fields that can serve as fuzzy membership functions (see Chapter 2). This approach is presented here and illustrated on a simple example.

The eSNN structure used for the presentation of the method is shown in Fig. 7 illustrated only on one input variable. Each input variable v_i is translated into trains of spikes and distributed to L1 neurons through delayed synaptic connections. The L2 neurons are created using an evolving learning algorithm and one-pass data propagation.

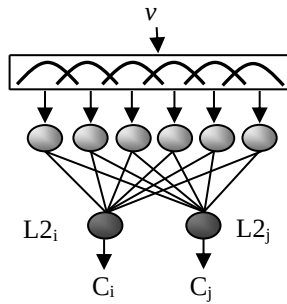


Fig.7. A simple structure of eSNN using only one input variable for the illustration of the method of fuzzy rule extraction

In this network, the input values of each input variable v_i are encoded using a family of m equally spaced overlapping Gaussian receptive fields [74] (RF) and distributed to multiple L1 neurons through m delayed synaptic connections. There is no delay at the centers of the Gaussian receptive fields and this delay increases towards the receptive field edges. Fig. 8 shows an example of population encoding of two input variables v_k and v_l , where $v_k < v_l$. Each input variable is encoded into a six-dimensional vector of spike times. The most stimulated L1 neuron fires first ($t_f = 0$) and the least stimulated neuron fires last ($t_f = t_{max}$). The values are deliberately chosen so that they cause a maximum excitation of two different L1 neurons, *i.e.* two earliest spikes appear at the $L1_{k3}$ and $L1_{l4}$ terminals at the same time, and two very similar spike patterns occur out of sequence by one L1 neuron. There is a relationship between which L1 neuron spikes first in a set of L1 neurons and the value encoded into this set of L1 neurons; when smaller values are encoded, the first neuron to spike tends towards the lower end of the observed set ($L1_{k1}$ and $L1_{l1}$ in this example) and when higher values are encoded the first neuron to spike tends towards the higher end of the observed set ($L1_{k6}$ and $L1_{l6}$ in this example)

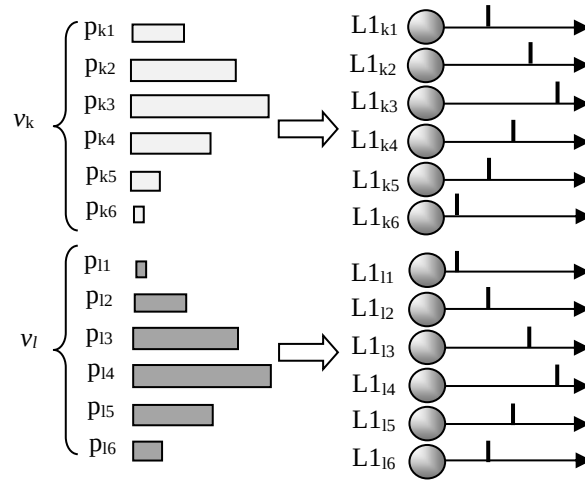


Figure 8. An example of population encoding of two input variables v_k and v_l ($v_k < v_l$) using six receptive fields ($m = 6$).

The spikes are propagated from the excitatory L1 neurons to the computationally simple L2 integrate-and-fire neurons [75] in a feed-forward manner. Each L2 neuron is a coincidence detection unit allowed to emit only one spike.

Each incoming spike from an L1 neuron influences the behaviour of the L2 neurons, changing their inner state (post synaptic potential or *PSP*). *PSP* of an L2 at time t depends on the firing order o_j of all its pre-synaptic neurons $L1_j$ as follows:

$$PSP_i(t) = \sum_j PSP_{ji} = \sum_j w_{ji} \times \text{mod}^{o_j} \quad (16)$$

where PSP_{ji} is the post-synaptic potential of neuron i , $\text{mod} \in (0, 1)$ is the modulation factor and w_{ji} is the efficacy (weight) of the synapse connecting the $L2_i$ and an $L1_j$ neurons. The firing order o_j depends on the timing of the spike emitted by $L1_j$ relative to the spikes emitted by other L1 neurons. A firing order $o_j = 0$ is assigned if neuron $L1_j$ spikes first amongst all L1 neurons. The L1 neuron that spikes second is assigned $o_j = 1$, and so on. The L1 neuron that spikes last is given $o_j = m-1$.

An L2 neuron fires a post-synaptic spike when its *PSP* reaches a certain threshold value PSP_θ . This model does not include the refractory period, therefore, immediately after firing the post-synaptic spike, the neuron's *PSP* returns to 0.

The synapses between L1 and L2 neurons are dynamic; their values change over the timescale of training. During training, connections conveying earlier spikes are given a higher weight resulting in a greater strengthening of connections:

$$\Delta w_{ji} = \text{mod}^o j \quad (17)$$

Hence, the synaptic connections conveying earlier spikes become more effective in causing the L2 neuron to fire in the future.

Each L1 neuron has a single connection to its input and one weighted synaptic connection to each of the L2 neurons denoted as a weight w_{ji} . The L2 layer is initially empty and all L2 neurons are built over time from the input data vectors by a fast evolving learning algorithm [68] that allows the learning of new input patterns without forgetting the older knowledge.

During training, a new neuron, $L2_n$, is created for a training pattern p_i , trained and its PSP_{θ_n} is set to a proportion of its maximum post synaptic potential value PSP_{nmax} . Thus, L2 neurons have dynamic thresholds and their selectiveness can be controlled by adjusting their PSP_{θ} .

Let the current pattern p_i belong to a class C_c . The Euclidean distances between $L2_n$ to all pre-existing $L2_o \in \{L2 | C=C_c\}$ are calculated and used as a similarity measure. If the similarity between $L2_n$ and $L2_o$ is below some threshold value S_0 ($S_0 > 0$) then $L2_n$ is aggregated with $L2_o$. During aggregation the weights and threshold of $L2_n$ are averaged into the values of $L2_o$, and $L2_n$ is discarded. Thus, the network's structure continuously evolves through the creation and merging of neurons based on incoming data. As the L2 layer is being created clusters of L2 neurons are formed. The L2 neurons from a cluster G_p are trained on the characteristics of only one class (positive examples). Hence, every $L2 \in G_p(C_p)$ learns to respond with earlier spikes when presented with input patterns from C_p .

Receptive fields not only increase the temporal distance between the input patterns, which improves the selectivity of the eSNN with rank order population coding, but it also opens the possibilities of using these networks for knowledge discovery. The eSNN builds a knowledge base that can be extracted during and after the learning process. These appear in the form of zero-order Takagi-Sugeno fuzzy rules:

$$\text{IF}(v_i \text{ is } F_i) \text{ AND } \dots (v_n \text{ is } F_n) \text{ THEN } (y \text{ is } C_k) \quad (18)$$

where v_i is an input variable, F_i is a linguistic value represented by its membership function such as SMALL, MEDIUM, LARGE *etc.*, and C_k represents a class label. The antecedent part in the rules is a composition, using the AND operators, of fuzzy conditions. The consequent part, the output, is a constant. The conditions, v_i is F_i , are found through an analysis of the connections w_{ji} between a pair of L1 and L2 neurons.

For example, the following two fuzzy rules can be extracted from a trained eSNN as explained below:

Rule0: IF (v_1 is SMALL) AND (v_2 is LARGE) AND (v_3 is LARGE)
THEN Class 1 (e.g. water₀)

Rule1: IF (v_1 is LARGE) AND (v_2 is LARGE) AND (v_3 is SMALL)
THEN Class 2 (e.g. water₁)

(19)

If for example, a sample of water gives the readings small for sensor 1 and large for sensors 2 and 3, then this sample is water₀ in accordance with the rules stated above. For the sample to be water₁, the readings would be large for sensors 1 and 2 and small for sensor 3.

To better explain the idea, let us consider a simple network shown in Fig.7 where two L2 neurons are created; a neuron L2_i has been trained to recognize the class C_i samples and a neuron L2_j has been trained to recognize the class C_j samples.

Let two one-dimensional vectors v_i and v_j belong to classes C_i and C_j, respectively. It is postulated that knowledge about the relationship between two inputs, v_i and v_j (e.g. that $v_i < v_j$), is stored in the synaptic weights of the L2 neurons representing a long-term memory. The change in synaptic weight between L1 and L2 neurons in the eSNN depends on the firing time of the L1, i.e. $\Delta w_{ji} = f(t_j^f)$. As stated earlier, the synaptic weights associated with the connections which convey earlier spikes increase more than those which convey later spikes. Hence, based on the weight patterns it is possible to deduce the size of input values relative to other input values (small, medium, large) and the contribution of this values to the modeled output.

An example, where two L2 neurons have evolved is explained below. L2_i has been trained to recognize the class C_i samples and L2_j has been trained to recognize the class C_j samples. Assume that at time t , a spike arrives from L1_n via two synaptic terminals (w_{ni} , w_{nj}). This spike excites the membrane potentials of both L2_i and L2_j as follows:

$$\Delta PSP_i = w_{ni} \times \text{mod}^{o_n} \quad (20)$$

$$\Delta PSP_j = w_{nj} \times \text{mod}^{o_n} \quad (21)$$

with the same convention as in (54) and where o_n is in the range $[0, m-1]$ and $m = 6$.

The difference between these two excitations is:

$$\begin{aligned} dPSP &= |PSP_i - PSP_j| = |\text{mod}^{o_n} \times (w_{ni} - w_{nj})| \\ &= f(w_{ni} - w_{nj}) = f(d_w). \end{aligned} \quad (22)$$

Hence the difference in the excitation of the two L2 neurons is a function of the difference between their synaptic weights (d_w). Thus, if two weights are identical, $d_w = 0$, both L2 neurons are equally excited by the incoming spike.

The d_w values for the theoretical pattern of the w_{ni} and w_{nj} weights in Fig. 9 is shown in Fig. 10. It can be seen that the L2_i neuron ‘favors’ lower input values, *i.e.* a lower input value will cause a bigger ΔPSP_i . Thus, lower values cause L2_i to spike before L2_j. The L2_j neuron does the opposite; it spikes before L2_i upon presentation of higher input values. Hence, the lower values v will be classified as C_i and the higher values as C_j as shown below and in Figs.30 and 31.

Rule 0: IF (v is SMALL) THEN C_i

Rule 1: IF (v is LARGE) THEN C_j (23)

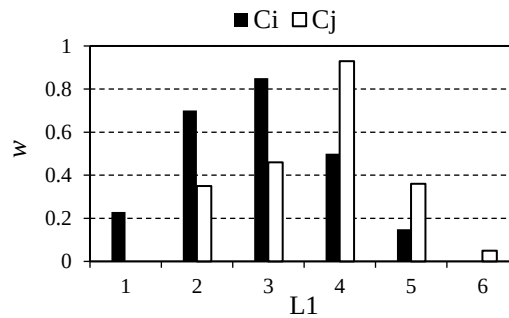


Figure 9. Two patterns of connection weights between levels L1 and L2 neurons representing the inputs from one input variable for class C_i and C_j samples

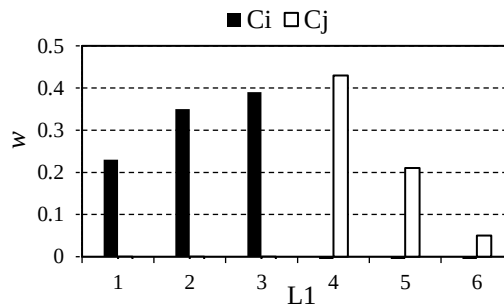


Figure 10. The difference of the connection weights d_w ($d_w = w_{ni} - w_{nj}$) for a single input variable for the two class patterns from Fig.9 shows a clear discrimination between class C_i and class C_j patterns that can be represented as fuzzy rules (23).

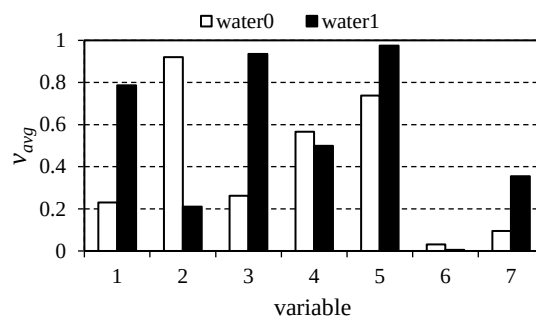
5.4.2. A case study of fuzzy rule extraction from water tastant sensory data

The data set samples were generated by an array of non-selective taste sensors based on conducting polymers that have been proven to be able to discriminate between basic tastants [79]. A data set D1 is comprised of 40 measured patterns from four different water types (two brands of mineral water, Milli-Q [80] water and distilled water), while a data set D2 contains 300 measured patterns from two brands of wine (Marcus James and Almadén Cabernet Sauvignon). Both sets are balanced, having an equal number of patterns of each type. The datasets were randomly shuffled and equally split into a training and testing set. To enable easier representation of the extracted rules only one L2 neuron per class has been evolved.

In each experiment, the eSNN model parameters (mod, C, Thr) (see chapter 4 for eSNN description), including the number of receptive fields m , have been carefully chosen for accurate taste recognition.

Firstly, the knowledge accumulated by an eSNN about the mineral waters (water₀, water₁) is given. The average values of the mineral water patterns in the training set are shown in Fig. 11. The samples have been obtained using seven taste sensors; hence each pattern is a seven-dimensional vector. The average values of v_6 in both mineral waters are significantly smaller than averages of the other variables. The difference between the v_6 values is smaller than in any of the other variables; this is represented within the rules obtained for these two water types as v_6 is omitted from classification criteria. The average values of all of the other variables show a considerable difference between the two mineral water types. For example, v_1 is much smaller in water₀ than in water₁, while v_2 is much smaller in water₁ than in water₀. This ‘prior knowledge’ is useful when the quality of knowledge representation is subsequently assessed.

Figure 12 shows the d_w values of an eSNN trained using the water₀ and water₁ patterns. In this case eight receptive fields ($m = 8$) have been used to encode the input patterns. As a result, each x_i feature pattern comprises of eight bars. However, if two weights are identical, $d_w = 0$, the corresponding bar will be missing in the feature pattern. This is observed in the pattern x_4 . Furthermore, the feature patterns are influenced by Gaussian receptive fields. Gaussian receptive fields increase the sparseness in the original data, *i.e.*, there is a smaller number of input neurons

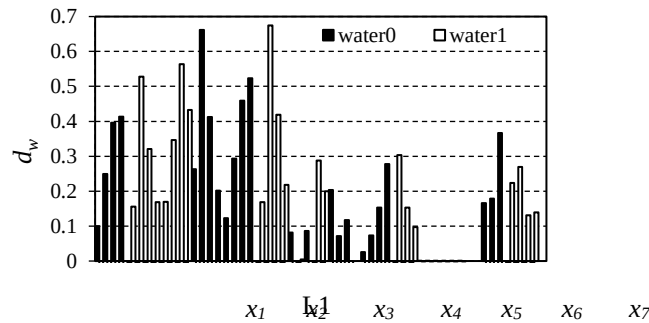


activated than without Gaussian receptive fields. As a result, only a subset of the connections convey spikes, others have weights equal to zero and their bars are missing from the pattern, as illustrated in x_7 . Also, some weight may be almost identical, therefore the small d_w will appear missing in the d_w plots.

Figure 11. Average values, v_{avg} , of the mineral waters' variables.

Figure 12. The d_w values of an eSNN trained to distinguish between mineral water patterns ($m = 8$).

It is interesting to observe that in the sixth feature pattern, x_6 , all d_w values are zero because the weights are similar. As a result the contributions of the spikes through these terminals to the post-synaptic potential of L2 neurons are very similar. It can therefore be deduced that the values of the



sixth input variable (with the rather small average values compared to other inputs) do not contribute to the process of distinguishing between the mineral water samples. Also, the spikes coming from the v_1 , v_2 and v_3 variables contribute more to the L2 neurons' *PSP*, than those coming from v_4 , v_5 and v_7 , indicating that these features are more important in the classification of the two mineral waters.

The following rules have been extracted for water₁ and water₂ samples:

Rule 0: IF (v_1 is SMALL) AND (v_2 is LARGE) AND (v_3 is SMALL) AND (v_4 is LARGE) AND (v_5 is SMALL) AND (v_7 is SMALL) THEN water₀

Rule 1: IF (v_1 is LARGE) AND (v_2 is SMALL) AND (v_3 is LARGE) AND (v_4 is SMALL) AND (v_5 is LARGE) AND (v_7 is LARGE) THEN water₁

For example, the above rules state that v_3 is SMALL in water₀ (Rule 0) and LARGE in water₁ (Rule1).

For the wine dataset D2 the following fuzzy rules have been extracted for the two brands of wine (wine₀, wine₁):

Rule 0: IF (v_1 is SMALL) AND (v_2 is LARGE) AND (v_3 is SMALL) AND (v_4 is SMALL) AND (v_5 is LARGE) AND (v_6 is LARGE) AND (v_7 is SMALL) THEN wine₀

Rule 1: IF (v_1 is LARGE) AND (v_2 is SMALL) AND (v_3 is SMALL) AND (v_4 is LARGE) AND (v_5 is LARGE) AND (v_6 is SMALL) AND (v_7 is LARGE) THEN wine₁

It is noteworthy that all input variables contribute to the classification of the wines, including v_6 which seems irrelevant in the classification of the water types. We contribute this to the fact that the v_6 averages for both wines are significant (Fig. 13).

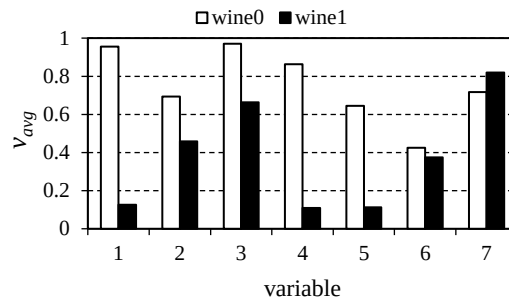


Figure 13. Average values, v_{avg} , of the wines' variables.

The two experiments described demonstrated knowledge discovery in a two-class problem scenario using eSNN. The knowledge extraction capability of the eSNN model on three-class and four-class problems has also been investigated. Firstly, the Milli-Q samples have been included in the experiment (water₂). The water₂ patterns have very small average values compared to the mineral water patterns (Fig. 14).

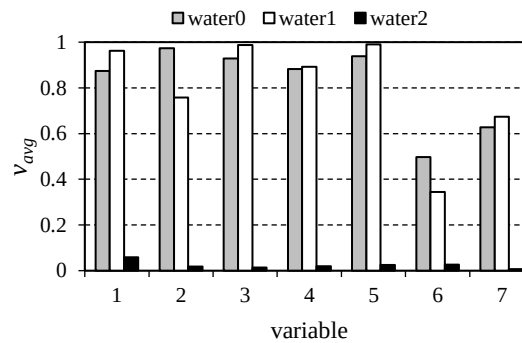


Figure 14. Average values, v_{avg} , of the two mineral waters and Milli-Q patterns.

Six receptive fields have been used ($m = 6$) resulting in six bars within each feature pattern. Again one L2 neuron per water type have been evolved. Their distance weight d_w values are shown in Fig. 15. It is interesting to see that while the average values of the water₂ samples are much smaller than

the average values of water_0 and water_1 , the weights of the L2 neuron that has learnt to recognize these samples are noticeably greater than the weights of the other neurons. This means that values of the water_2 patterns were encoded in such a way that they have been repeatedly enhancing the same set of connections. This results in the considerable observed strengthening of the connections.

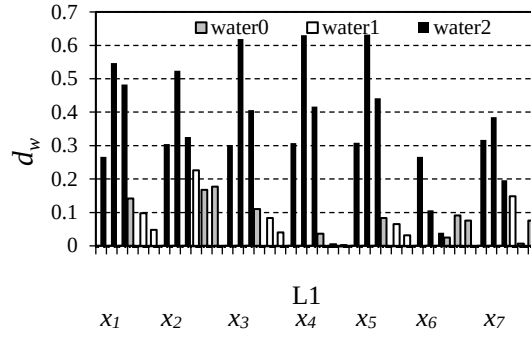


Figure 15. The d_w values of an eSNN trained to distinguish between three waters ($m = 6$).

Contrary to this, the mineral water samples have more evenly distributed their influences among a larger number of synaptic connections. This is due to the fact that the values in the water_0 and water_1 patterns are very similar and they compete for the same set of connections. This is also reflected in the extracted rules ('1' indicates SMALL, '2' indicates MEDIUM and '3' indicates LARGE):

Rule 0: IF (v_1 is 2) AND (v_2 is 3) AND (v_3 is 2) AND (v_4 is 2) AND (v_5 is 2) AND (v_6 is 3) AND (v_7 is 3) THEN water_0

Rule 1: IF (v_1 is 3) AND (v_2 is 2) AND (v_3 is 3) AND (v_4 is 3) AND (v_5 is 3) AND (v_7 is 2) THEN water_1

Rule 2: IF (v_1 is 1) AND (v_2 is 1) AND (v_3 is 1) AND (v_4 is 1) AND (v_5 is 1) AND (v_7 is 3) THEN water_2

The approach has also been tested using all four water types (*i.e.*, two brands of mineral water, Milli-Q [80] water and distilled water (water_3)) with following characteristics:

- a similarity between the water_0 and water_1 averages and between the water_2 and water_3 averages,
- a difference in magnitude between water_2 and water_3 averages and the water_0 and water_1 averages.

Again only one neuron per water type has been evolved and the fuzzy rules using three membership functions have been extracted as follows:

Rule 0: IF (v_1 is 2) AND (v_2 is 3) AND (v_3 is 2) AND (v_4 is 2) AND (v_5 is 2) AND (v_6 is 2)

THEN water₀

Rule 1: IF (v_1 is 3) AND (v_2 is 2) AND (v_3 is 3) AND (v_4 is 3) AND (v_5 is 3) AND (v_6 is 2) AND (v_7 is 2) THEN water₁

Rule 2: IF (v_1 is 1) AND (v_2 is 1) AND (v_3 is 1) AND (v_4 is 2) AND (v_5 is 2) AND (v_7 is 1) THEN water₂

Rule 3: IF (v_1 is 2) AND (v_2 is 2) AND (v_3 is 2) AND (v_4 is 1) AND (v_5 is 1) AND (v_6 is 1) AND (v_7 is 2) THEN water₃

The degree of granularity of the fuzzification has been increased from three to four where, for instance, ‘1’ indicates VERY SMALL, ‘2’ indicates SMALL, ‘3’ indicates MEDIUM and ‘4’ indicates LARGE:

Rule 0: IF (v_1 is 3) AND (v_2 is 4) AND (v_3 is 3) AND (v_4 is 3) AND (v_5 is 3) AND (v_6 is 2) THEN water₀

Rule 1: IF (v_1 is 4) AND (v_2 is 3) AND (v_3 is 4) AND (v_4 is 4) AND (v_5 is 4) AND (v_6 is 3) AND (v_7 is 3) THEN water₁

Rule 2: IF (v_1 is 1) AND (v_2 is 1) AND (v_3 is 1) AND (v_4 is 2) AND (v_5 is 2) AND (v_7 is 1) THEN water₂

Rule 3: IF (v_1 is 2) AND (v_2 is 2) AND (v_3 is 2) AND (v_4 is 1) AND (v_5 is 1) AND (v_6 is 1) AND (v_7 is 2) THEN water₃

The resulting set of rules more accurately captures the data set characteristics. The high variable values that characterize the water₀ and water₁ patterns and their fine differences are described in Rule 0 and Rule 1, while the characteristics of the water₂ and water₃ values are embedded in Rule 2 and Rule 3.

The task of determining the number of L2 neurons is an important step in tuning the eSNN accuracy [81]. One L2 neuron per class might not be enough to achieve the best accuracy. As the number of neurons and rules increases it may become cumbersome to interpret these rules. This could become a problem if an eSNN is allowed to evolve in an open problem space.

5.5. Evolving SNN for reservoir computing

The eSNN principles presented and illustrated so far can be used to create evolving output classification or regression modules of reservoir types of SNN computational architectures which is the topic of this section.

5.5.1. Reservoir architectures

Two major types of reservoir computing is liquid state machine (LSM) [45] and echo state network [82]. LSM consists of many randomly interconnected recurrent neurons where each neuron receives input from other neurons in different times. This computational model is inspired from the idea of water ripples (output), which are generated after certain objects (input) are dropped on the still water. In an ideal setting, LSM that is constructed with a precise mathematical framework, promises universal computational power, for real-time computing on analogue function in continuous time. LSM is characterized as a model for adaptive computational system, which provides a method for employing randomly connected circuits, a theoretical context where various processors increase the computational power of a circuit and a method for multiplexing different computations within the same circuit [83].

During LSM simulation, the synaptic weights, neurons connectivity and neurons parameters are predefined and predetermined. Referring to Fig. 16, the continuous stream of input (e.g. trains of spikes) $u(t)$ is transmitted into liquid and will cause the neurons to respond and generate the liquid activity. The state of the liquid $x(t)$ that can be recorded at different time points is simply the current output of some operator or filter that maps input functions $u(t)$ onto function $x(t)$. This state is then passed to the memory-less readout function f that will transform into output $v(t)$.

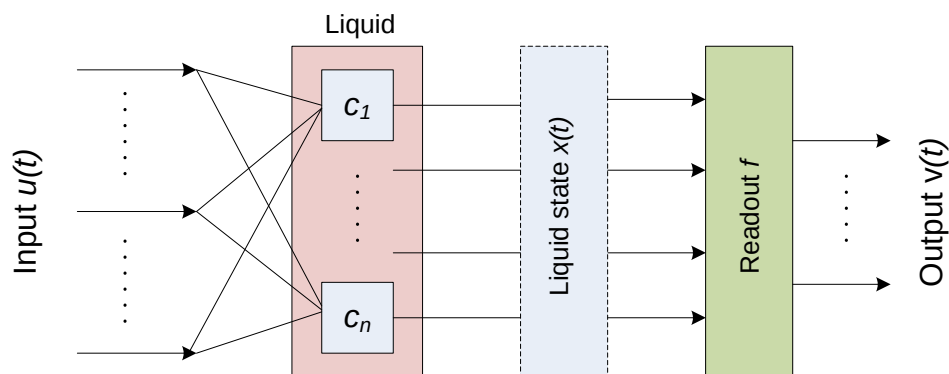


Figure 16. The LSM architecture consists of three main layers: input neuron layer, liquid state layer and readout function layer. eSNN and deSNN can be used as classifiers or regressors in the Readout module.

The connectivity in a LSM is usually pre-selected as a random connections or fully connected neuronal structures. As a special case, a small world connectivity (SWC) can be applied (fig. 17), where neuron a is connected to other neuron b with a probability $p_{a,b}$ that depends on the closeness of the two neurons. The closer they are (the smaller the distance between them $D_{a,b}$), the higher the probability to connect them. This is a biologically plausible connectivity rule as the brain is a small world connectivity system :

$$p_{a,b} = C \times e^{-D_{a,b}^2 / \lambda^2} \quad (24)$$

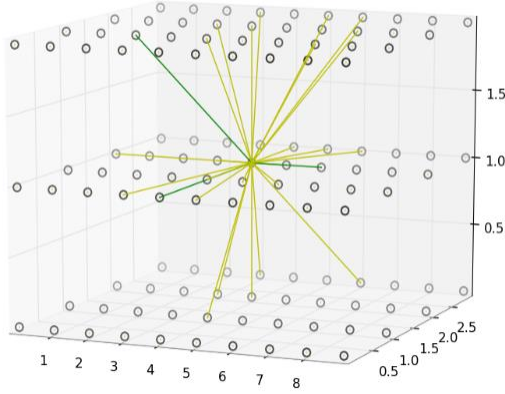


Fig.17. An example of a spiking neuron connectivity in a 3D reservoir of spiking neurons using the small-world connectivity rule.

An important characteristic of SNN is the separability of patterns that are learned.

Separability basically determines how well the liquid separates different classes of input into different reservoir states, or state vectors, and is similar to supervised clustering. According to [89], separability can be measured by dividing a set of states, $O(t)$, into N subsets, $O_m(t)$ for every class. Here N denotes the total number of classes and t represents the current iteration of the reservoir. The inter-class distance, $C_d(t)$, and intra-class variance, $C_v(t)$, $C_d(t)$ can be defined as:

$$C_d(t) = \frac{\sum_{m=1}^N \sum_{n=1}^N \|\mu(O_m(t)) - \mu(O_n(t))\|_2}{N^2} \quad (25)$$

$$C_v(t) = \frac{1}{N} \sum_{m=1}^N \rho(O_m(t)) \quad (26)$$

where, $\mu(O_m(t))$ is the center of mass for each class, $\rho(O_m(t))$ is average variance from each state within class m and can be calculated by

$$\mu(O_m(t)) = \frac{\sum_{O_n \in O_m(t)} O_n}{|O_m(t)|} \quad (27)$$

$$\rho(O_m(t)) = \frac{\sum_{O_n \in O_m(t)} \|\mu(O_m(t)) - O_n\|_2}{|O_m(t)|} \quad (28)$$

Therefore, based on Eq.26 and Eq.27, the separability of the liquid ($Sep \Psi$) for a set of states $O(t)$ can be calculated as [89]:

$$Sep_{\Psi}(O(t)) = \frac{C_d(t)}{C_v(t) + 1} \quad (29)$$

Using different types of probabilistic neurons as discussed in Chapter 4, can result in different separability of patterns in the reservoir activated by different stimuli as shown in Fig. 18 and experimented in [90].

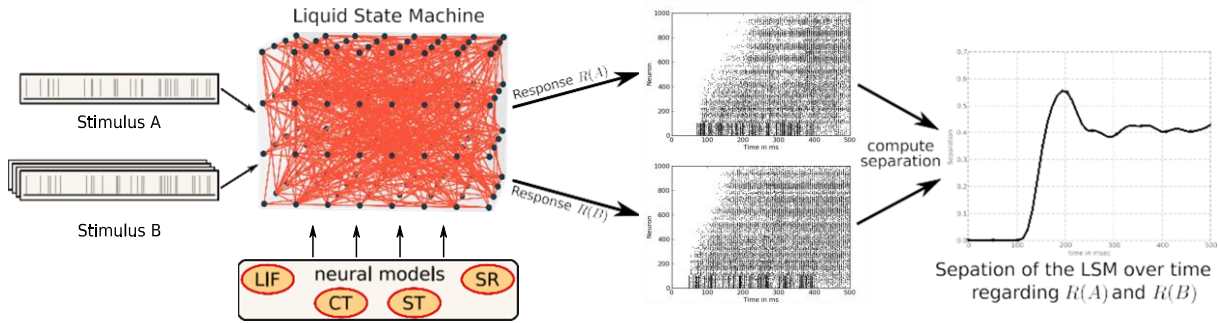


Fig. 18. Experimental setup of the study of using different probabilistic models of spiking neurons in a LSM in regards of separability of patterns activated by different spiking stimuli sequences. In [90] a comparative analysis is performed between LSM when different types of spiking neuron models are used, such as LIF; CT (continuously changing spiking threshold); ST (step-wise threshold); SR (stochastic reset) (see Chapter 4).

In regards to learning in the reservoir, there two types of reservoir architectures:

- SNN LSM, that involve no learning in the reservoir (the discussed so far above);
- SNN reservoir architectures that involve learning. Such architecture is NeuCbe discussed in the next chapter.

5.5.2. eSNN/deSNN as classification/regression systems for reservoir architectures

All reservoir architectures involve learning in the output classification or regression module (the read-out function as per Fig.17. This module is capable of generating different responses from different dynamic reservoir input patterns (separation property) that depends on the complexity of the liquid. In addition, it also has ‘approximation property’ that depends on the adaptation ability of the readout function that can distinguish responses, their generalization and relation with the given output [84]. Any statistical analysis or classifier can be used to define the readout function. As the inputs received can be in the form of continuous stream of data, it can be used to solve spatiotemporal problems such as in patterns recognition [85, 86], optimization [87] and classification problem [88].

Here we discuss how eSNN and deSNN can be used efficiently as output classification/regression modules for fast, evolving and meaningful supervised learning and pattern recognition of the patterns that are activated in the reservoir from input data.

The input-output mapping in the supervised learning can be imposed by means of error E calculation, defined as

$$E = \frac{1}{2} \sum_j (t_j^{\text{out}} - t_j^{\text{des}})^2 \quad (30)$$

where, E represents the difference between all network outputs t_j^{out} and desired outputs t_j^{des} . The weights w_{ij}^k is adjusted accordingly, so as to minimize the error E and can be expressed as

$$\Delta w_{ij}^k = -\eta \frac{dE}{dw_{ij}^k} \quad (31)$$

where, η denotes the learning rate. However, in the process of implementing this model, there are several research questions that arise and needs to be addressed. These research questions are mentioned in the following section.

Fig.19 shows a reservoir computational architecture where a deSNN model is used as a classifier of the spatio-temporal patterns in the reservoir.

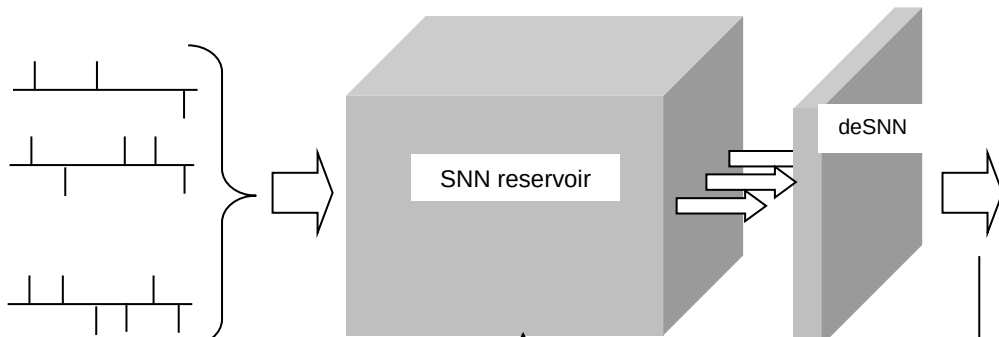


Figure 19. A reservoir computational architecture where a deSNN is used as a classifier of the dynamic patterns in the reservoir that are activated by different stimuli. A feedback from the deSNN output back to the reservoir can be established as well.

The next chapter develops further the architecture from Fig.19 and introduces a brain-like learning reservoir architecture that has as an output function deSNN.

5.6. Chapter summary and further readings for deeper knowledge

This chapter presents the concept and some implementations of evolving SNN. The methods of evolving SNN add new properties to the concept of SNN, namely:

- Fast data processing, possibly one iteration of data presentation only;
- Adaptive learning, that addresses the “curse of catastrophic forgetting”;
- Rule extraction, that addresses “the curse of NN black boxes”;
- Integration of different modalities of data, all represented as spike sequences.

eSNN extend the principles of ECOS from Chapter 2 using spike information representation and spike-time learning. At the same time they keep all principles of ECOS that are knowledge based ANN. The rules extracted from eSNN represent ‘flat’ knowledge, even thou they can capture some temporal characteristics from data.

eSNN, including deSNN models are used as classifiers for brain-inspired SNN presented in Chapter 6, where deep learning and deep knowledge representation are facilitated.

Readings on specific topics of eSNN and deSNN can be found as follows:

- Rule extraction from eSNN [72];
- deSNN [41];
- Computational modelling with SNN (chapter 37 in [92]);
- Brain-like information processing for spatio-temporal pattern recognition (chapter 47 in [92]);

- Probabilistic SNN for reservoir computing [90];
- On the role of time and space in neural computation [32];
- Liquid State Machines [83];
- Overview of evolving connectionist systems ([91]).

Acknowledgements

Parts of the material in this chapter have been published previously as referenced in the corresponding sections. I acknowledge the contribution of my co-authors in these publications: L.Benuskova, S.Wysocki, S.Schliebs, S. Soltic, A. Mohemmed, K.Double, N.Nuntalid, G.Indiveri.

References

- [1] W. Gerstner and W. M. Kistler, (2002), *Spiking Neuron Models: Single Neurons , Populations , Plasticity*. Cambridge University Press, 2002.
- [2] L. M. Optican and B. J. Richmond (1987), Temporal encoding of two-dimensional patterns by single units in primate inferior temporal cortex. III. Information theoretic analysis.pdf. *Journal of Neurophysiology*, Vol. 57, No. 1, 1987, 162–177.
- [3] R. Lestienne (1996), Determination of the precision of spike timing in the visual cortex of anaesthetised cats. *Biological Cybernetics*, Vol. 74, 1996, 55–61. doi:10.1007/BF00199137
- [4] Z. F. Mainen and T. J. Sejnowski (1995), Reliability of spike timing in neocortical neurons. *Science*, 268(5216), 1995, 1503–1506.
- [5] S. Thorpe, D. Fize and C. Marlot (1996), Speed of processing in the human visual system. *Nature*, 381(6582), 520–522.
- [6] S. Thorpe and J. Gautrais (1998), Rank Order Coding. *Computational Neuroscience: Trends in Research*, 113–119.
- [7] S. M. Bohte, H. La Poutre and J. N. Kok (2002), Unsupervised Clustering With Spiking Neurons By Sparse Temporal Coding and Multilayer RBF Networks. *IEEE Transactions on Neural Networks*, Vol. 13. No. 2, 426–435.
- [8] H. N. Abdul Hamed (2012), *Novel Integrated Methods of Evolving Spiking Neural Network and Particles Swarm Optimazation*, Auckland University of Technology.
- [9] S. Schliebs, M. Defoin-Platel and N. Kasabov (2009), Integrated feature and parameter optimization for an evolving spiking neural network. *Advances in Neuro-Information*, 1229–1236. Retrieved from http://link.springer.com/chapter/10.1007/978-3-642-02490-0_149

- [10] N. Kasabov, V. Feigin, Z.-G. Hou, Y. Chen, L. Liang, R. Krishnamurthi, ... and P. Parmar, (2014), Evolving Spiking Neural Networks for Personalised Modelling, Classification and Prediction of Spatio-Temporal Patterns with a Case Study on Stroke. *Neurocomputing*, Vol. 134, 269–279. doi:10.1016/j.neucom.2013.09.049
- [11] K. Dhoble, N. Nuntalid, G. Indiveri, and N. Kasabov (2012), Online spatio-temporal pattern recognition with evolving spiking neural networks utilising address event representation, rank order, and temporal spike learning. *The 2012 International Joint Conference on Neural Networks (IJCNN)*, 1–7. doi:10.1109/IJCNN.2012.6252439
- [12] S. Thorpe, A. Delorme and R. Van Rullen (2001), Spike-based strategies for rapid processing. *Neural Networks : The Official Journal of the International Neural Network Society*, Vol. 14(6-7), 715–25. Retrieved from <http://www.ncbi.nlm.nih.gov/pubmed/11665765>.
- [13] S. G. Wysoski, L. Benuskova and N. Kasabov (2006), On-Line Learning with Structural Adaptation in a Network of Spiking Neurons for Visual Pattern, 61–70.
- [14] H. N. Abdul Hamed, N. Kasabov, Z. Michlovský and S. M. Shamsuddin (2009), String Pattern Recognition Using Evolving Spiking Neural Networks and Quantum Inspired Particle Swarm Optimization (pp. 611–619). Springer Berlin Heidelberg.
- [15] S. G. Wysoski, L. Benuskova N. Kasabov (2007b), Text-Independent Speaker Authentication with Spiking Neural Networks, 758–767.
- [16] S. G. Wysoski, L. Benuskova and N. Kasabov (2007a), Spiking Neural Networks for Text-Independent Speaker Authentication. In *Artificial Neural Networks–ICANN* (Vol. 2, pp. 758–767). Springer Berlin Heidelberg.
- [17] S. M. Bothe, H. La Poutre and J. N. Kok (2002), Unsupervised clustering with spiking neurons by sparse temporal coding and multilayer RBF networks, *IEEE Transactions on Neural Networks*, Vol. 13, 2002, 426–435.
- [18] J. Gautrais and S. Thorpe (1998), Rate coding versus temporal order coding: a theoretical approach, *BioSystems*, Vol. 48, 1998, 57–65.
- [19] B. Schrauwen and J. Van Campenhout (2003), BSA, a fast and accurate spike train encoding scheme, *IEEE, In: Proceedings of the International Joint Conference on Neural Networks*. Vol. 4, 2003, 2825–2830.
- [20] A. L. Hodgkin and A. F. Huxley (1952), A quantitative description of membrane current and its application to conduction and excitation in nerve, 117, 500–544.
- [21] M. Nelson and J. & Rinzel (1995), The Hodgkin-Huxley Model. In J. M. Bower & Beeman (Eds.), *The book of Genesis* (pp. 27–51). New York: Springer.

- [22] C. Meunier and I. Segev (2002), Playing the Devil's advocate : is the Hodgkin – Huxley model useful? *Trends in Neurosciences*, Vol. 25, No. 11, 2002, 558–563.
- [23] A. F. Strassberg and L.J DeFelice (1993), Limitation of the Hodgkin-Huxley Formalism: Effects of Single Channel Kinetics on Transmembrane Voltage Dynamics, *MIT Press, Neural Computation*, Vol.5, No. 6, 1993, pp. 843-855.
- [24] F. Bezanilla and C. M. Armstrong (1997), Inactivation of the Sodium Channel. I. Sodium Current Experiments, *Journal of General Physiology*, Vol. 40, No. 5, 1997, 549-566.
- [25] L. Lapicque (1907), Recherches quantitatives sur l'excitation électrique des nerfs traitée comme une polarisation. *Physiol Pathol Gen*, Vol. 9, No. 1, 1907, 620–635. doi:10.1007/s00422-007-0189-6.
- [26] E. M. Izhikevich (2003), Simple model of spiking neurons, *IEEE Transactions on Neural Networks*, Vol. 14, 2003, 1569-1572.
- [27] W. Gerstner and J. L. van Hemmen (1992), Associative Memory in a Network of “Spiking” Neurons. *Network: Computation in Neural Systems*, Vol. 3, No. 2, 1992, 139–164.
- [28] W. Gerstner (1998), Spiking Neurons, In W. Maass & C. M. Bishop (Eds.), *Pulsed Neural Networks* (pp. 3–54). MIT Press.
- [29] A. Delorme and S. J. Thorpe (2003), SpikeNET : an event-driven simulation package for modelling large networks of spiking neurons. *Network: Computation in Neural Systems*, Vol. 14, 2003, 613–627.
- [30] N. Kasabov (2010), To spike or not to spike: A probabilistic spiking neural model, *Neural Networks*, Vol. 23, No. 1, 2010, 16-19.
- [31] S. M. Bohte (2004), The evidence for neural information processing with precise spike-times: A survey. *Natural Computing*, Vol. 3, No. 2, 2004, 195–206. doi:10.1023/B:NACO.0000027755.02868.60
- [32] W. Maass (1998), On the role of time and space in neural computation. *Mathematical Foundations of Computer Science 1998*, 72–83.
- [33] A. Kasinski and F. Ponulak (2006), Comparison of Supervised Learning Methods for Spike Time. *International Journal of Applied Mathematics and Computer Science*, Vol. 16, No. 1, 2006, 101–113.
- [34] D. E. Rumelhart G. E. Hinton and R. J. Williams (1986), Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- [35] S. M. Bohte, J. N. Kok and H. La. Poutre (2000), SpikeProp : Backpropagation for Networks of Spiking Neurons Error-Backpropagation in a Network of Spiking Neurons. *ESANN*, 419–424.

- [36] C. C. Bell, V. Z. Han, Y. Sugawara and K. Grant (1997), Synaptic plasticity in a cerebellum-like structure depends on temporal order. *Nature*, Vol. 387, 1997, 278–281. doi:10.1038/387278a0
- [37] G. Bi and M. Poo (1998), Synaptic Modifications in Cultured Hippocampal Neurons : Dependence on Spike Timing , Synaptic Strength , and Postsynaptic Cell Type. *The Journal of Neuroscience*, Vol. 18, No. 24, 1998, 10464–10472.
- [38] H. Markram, J. Lubke, M. Frotscher and B. Sakmann (1997), Regulation of Synaptic Efficacy by Coincidence of Postsynaptic APs and EPSPs. *Science*, 275(January), 213–215.
- [39] T. Masquelier, R. Guyonneau and S. J. Thorpe (2008), Spike timing dependent plasticity finds the start of repeating patterns in continuous spike trains, *PLoS ONE*, Vol. 3, No. 1:e1377.
- [40] S. Fusi, M. Annunziato, D. Badoni, A. Salamon and D. J. Amit (1999), Spike-driven synaptic plasticity: theory, simulation, VLSI implementation. *Neural Computation*, Vol. 12, No. 10, 1999, 2227–2258. doi:10.1162/089976600300014917
- [41] N. Kasabov, K. Dhoble, N. Nuntalid and G. Indiveri (2013), Dynamic evolving spiking neural networks for on-line spatio- and spectro-temporal pattern recognition. *Neural Networks : The Official Journal of the International Neural Network Society*, 41(1995), 188–201. doi:10.1016/j.neunet.2012.11.014
- [42] S. Mitra, S. Fusi and G. Indiveri (2009), Real-Time Classification of Complex Patterns Using Spike-Based Learning in Neuromorphic VLSI. *IEEE Transactions on Biomedical Circuits and Systems*, Vol. 3, No. 1, 2009, 32–42. doi:10.1109/TBCAS.2008.2005781
- [43] M. Tsodyks, K. Pawelzik and H. Markram (1998), Neural Networks with Dynamic Synapses, *Neural Comput.*, Vol. 10, No. 4, 1998, 821–835.
- [44] W. Maass and E. D. Sontag (2000), Neural Systems as Nonlinear Filters, *Neural Comput.*, Vol. 12, No. 8, 2000, 1743–1772.
- [45] W. Maass, T. Natschlager and H. Markram (2002), Real-Time Computing Without Stable States: A New Framework for Neural Computation Based on Perturbations. *Neural Computation*, Vol. 14, No. 11, 2002, 2531–2560.
- [46] T. Natschläger and W. Maass (2002), Spiking neurons and the induction of finite state machines, *Theoretical Computer Science - Natural Computing*, Vol. 287, No. 1, 2002, pp.251-265.
- [47] H. Namarvar, J.-S. Liaw and T. Berger (2001), A new dynamic synapse neural network for speech recognition, In *Neural Networks, 2001.Proc. IJCNN '01. Int. Joint Conf. on*, 2001.
- [48] N. Mehrtash, D. Jung, and H. Klar (2003), Image pre-processing with dynamic synapses. *Neural Computing & Applications*, 12:33–41, 2003.10.1007/s00521-030-0371-2.

- [49] A. Mohemmed, S. Schliebs, S. Matsuda, K. Dhoble and N. Kasabov (2011), Optimization of spiking neural networks with dynamic synapses for spike sequence generation using pso. In International Joint Conference on Neural Networks. IEEE Publishing, San Jose, California, USA (2011).
- [50] A. Mohemmed, S. Schliebs and N. Kasabov (2011), Method for training a spiking neuron to associate input output spike trains. In Engineering Applications of Neural Networks. Springer, Corfu, Greece (2011). In Print.
- [51] M. C. van Rossum (2001), A novel spike distance. *Neural Computation*, Vol. 13, No. 4, 2001, 751–763.
- [52] F. Ponulak (2005), ReSuMe – new supervised learning method for spiking neural networks. Tech. rep., Institute of Control and Information Engineering, Poznań University of Technology, Poznań, Poland (2005).
- [53] R. V. Florian (2010), The chronotron: a neuron that learns to fire temporally-precise spike patterns. <http://precedings.nature.com/documents/5190/version/1> (2010).
- [54] W. Maass, T. Natschläger and H. Markram (2002), Realtime computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14(11) (2002) 2531–2560.
- [55] R. Gutig and H. Sompolinsky (2006), The tempotron: a neuron that learns spike timing-based decisions. *Nat Neurosci*, Vol. 9, No. 3, 2006, 420–428.
- [56] W. Gerstner and W. M. Kistler (2002), *Spiking Neuron Models: Single Neurons, Populations, Plasticity*, Cambridge University Press, Cambridge, MA (2002).
- [57] E. Nordlie, M.-O. Gewaltig and H. E. Plesser (2009), Towards reproducible descriptions of neuronal network models. *PLoS Comput Biol*, Vol. 5, No. 8, 2009, e1000456.
- [58] M.-O. Gewaltig and M. Diesmann (2007), Nest (neural simulation tool), *Scholarpedia*, Vol. 2, No. 4, 2007, 1430.
- [59] F. Ponulak and A. Kasin'ski (2010), Supervised learning in spiking neural networks with ReSuMe: sequence learning, classification, and spike shifting. *Neural Computation*, Vol. 22, No. 2, 2010, 467–510. PMID: 19842989.
- [60] W. Maass, T. Natschläger and H. Markram (2002), Realtime computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, Vol. 14, No. 11, 2002 2531–2560.
- [61] B. Schrauwen and J. V. Campenhout (2007), Linking nonbinned spike train kernels to several existing spike train metrics. *Neurocomput.*, Vol. 70, 2007, 1247–1253.

- [62] J. D. Victor and K. P. Purpura (1997), Metric-space analysis of spike trains: theory, algorithms and application. *Network: Computation in Neural Systems*, Vol. 8, No. 2, 1997, 127–164.
- [63] M. C. van Rossum (2001), A novel spike distance, *Neural Computation*, Vol. 13, No. 4, 2001, 751–763.
- [64] G. Kistler and W. Gerstner (2002), *Spiking Neuron Models - Single Neurons, Populations, Plasticity*, Cambridge Univ. Press, 2002.
- [65] P. Lichtsteiner and T. Delbruck (2005), A 64x64 aer logarithmic temporal derivative silicon retina, *Research in Microelectronics and Electronics*, 2005 PhD, Vol. 2, No. 1, 202–205. doi:10.1109/RME.2005.1542972.
- [66] T. Delbruck (2007), jAER open source project, 2007, <http://jaer.wiki.sourceforge.net>
- [67] S. Soltic and N. Kasabov (2010), Knowledge extraction from evolving spiking neural networks with rank order population coding. *Int. J. Neural Syst.*, Vol. 20, No. 6, 2010, 437–445.
- [68] N. Kasabov 2007, *Evolving Connectionist Systems: The Knowledge Engineering Approach*, Springer-Verlag, 2nd edition; 1st edition - 2002.
- [69] S. Wysoski, L. Benuskova and N. Kasabov (2010), Evolving Spiking Neural Networks for Audiovisual Information Processing, *Neural Networks*, Vol. 23, No. 7, 2010, 819-835.
- [70] S. Schliebs and N. Kasabov (2012), *Evolving spiking neural networks: A Survey*, Evolving Systems, Springer, 2012.
- [71] S. Thorpe, A. Delorme and R. Van Rullen (2001), Spike-based strategies for rapid processing, *Neural Networks*, Vol. 14, 2001, 715-725.
- [72] S. Soltic and N. K. Kasabov (2010), Knowledge extraction from evolving spiking neural networks with rank order population coding. *Int. J. Neural Syst.*, Vol. 20, No. 6, 2010, 437–445.
- [73] S. P. Johnston, G. Prasad, L. Maguire and T. M. McGinnity (2010), An FPGA hardware/software co-design towards evolvable spiking neural networks for robotics application. *International Journal of Neural Systems*, Vol. 20, No. 6, 2010, 447–461.
- [74] P. Tiesinga, J. Fellous and T. J. Sejnowski (2008), Regulation of spike timing in visual cortical circuits. *Nat Rev Neurosci*, Vol. 9, No. 2, 2008, 97–107.
- [75] E. Nichols, L. J. McDaid and N. H. Siddique (2010), Case study on a self-organizing spiking neural network for robot navigation. *International Journal of Neural Systems*, Vol. 20, No. 6, 2010, 501–508. PMID: 21117272.
- [76] S. J. Thorpe, R. Guyonneau, N. Guilbaud, J.-M. Allegraud and R. VanRullen: SpikeNet (2004), real-time visual processing with one spike per neuron. *Neurocomputing*, 58-60, 2004, 857 – 864.

- [77] L. Bako (2010), Real-time classification of datasets with hardware embedded neuromorphic neural networks. *Briefings in Bioinformatics*, Vol. 1, No. 3, 2010, 348 –363.
- [78] S. M. Bohte, J. N. Kok and J. A. L. Poutr'e (2000), SpikeProp: backpropagation for networks of spiking neurons. In *ESANN*, 419–424, 2000.
- [79] A. Riul Jr, D. S. dos Santos Jr, K. Wohnrath, R. Di Tommazo, A. C. P. L. F. Carvalho, F. J. Fonseca, O. N. Oliveira Jr, D. M. Taylor and L. H. C. Mattoso (2002), An artificial taste sensor: efficient combination of sensors made from Langmuir-Blodgett films of conducting polymers and a ruthenium complex and self-assembled films of an Azobenzene-containing polymer, *Langmuir*, Vol. 18, 2002, 239-245.
- [80] Milli-Q, <http://www.millipore.com/>
- [81] S. Soltic, S. G. Wysoski and N. Kasabov (2008), Evolving spiking neural networks for taste recognition, in *Proc. Int. Joint Conf. on Neural Networks IJCNN2008*, (Hong Kong, 2008), pp. 2092-2098.
- [82] Jaeger, H., & Haas, H. (2004), Harnessing nonlinearity: predicting chaotic systems and saving energy in wireless communication. *Science* (New York, N.Y.), 304(5667), 78–80. doi:10.1126/science.1091277
- [83] W. Maass (2010), Chapter 1 Liquid State Machines : Motivation , Theory , and Applications.
- [84] B. J. Grzyb, E. Chinellato, G. M. Wojcik and W. A. Kaminski (2009), Which Model to Use for the Liquid State Machine ? In *Neural Networks, 2009. IJCNN 2009. International Joint Conference on* (pp. 1018–1024). IEEE.
- [85] E. Goodman and D. Ventura (2006), Spatiotemporal Pattern Recognition via Liquid State Machines. In *IJCNN* (pp. 3848–3853).
- [86] S. Schliebs, N. Nuntalid and N. Kasabov (2010), Towards Spatio-Temporal Pattern Recognition Using Evolving Spiking Neural Networks. In *Neural Information Processing. Theory and Algorithms*, Vol. 6443, pp. 163–170. doi:10.1007/978-3-642-17537-4_21
- [87] Z. Yanduo and W. Kun (2009), The Application of Liquid State Machines in Robot Path Planning, *Journal of Computers*, Vol. 4 No. 11), 1182–1186.
- [88] H. Ju, J. Xu and A. M. J. VanDongen (2010), Classification of Musical Styles Using Liquid State Machines. *Neural Networks (IJCNN)*, The 2010 International Joint Conference, 1–7.
- [89] D. Norton and D. Ventura (2009), Improving the Separability of a Reservoir Facilitate Learning Transfer, *Proceeding of the Seventh ACM Conference on Creativity and Cognition*, 339-340.
- [90] Schliebs, S., Mohemmed, A., & Kasabov, N., Are Probabilistic Spiking Neural Networks Suitable for Reservoir Computing? In *International Joint Conference on Neural Networks*, pp. 3156-3163. San Jose, USA, 2011

In: N. K. Kasabov, *Time-Space, Spiking Neural Networks and Brain- Inspired Artificial Intelligence*, Springer-Verlag GmbH Germany part of Springer Nature, 2019, Springer Series on Bio- and Neurosystems 7, https://link.springer.com/chapter/10.1007/978-3-662-57715-8_5

[91] N. Kasabov (2015) Evolving connectionist systems: From neuro-fuzzy-, to spiking – and neurogenetic, in: Kacprzyk and Pedrycz (eds) Springer Handbook of Computational Intelligence, Springer, 771-782.

[92] N.Kasabov (ed) (2014) Springer Handbook of Bio-/Neuroinformatics, Springer, 2014.